Architectures CNN avancées

3.1 Rappel : Fonctionnement des CNN

Pourquoi les CNN pour la vision ?

- **Problème des réseaux fully-connected** :
  - Image 224x224x3 = 150,528 pixels
  - Première couche FC de 1000 neurones = 150 millions de paramètres
  - Pas d'exploitation de la structure spatiale de l'image
- **Intuition des CNN** :
  - Les motifs visuels sont **locaux** (bords, textures, formes)
  - Les motifs sont **invariants par translation** (un chat reste un chat peu importe sa position)
  - Hiérarchie de complexité : pixels -> bords -> textures -> parties -> objets

Les 3 opérations fondamentales

1. Convolution

Principe : appliquer un filtre (kernel) qui "glisse" sur l'image

```
Image d'entrée          Filtre 3x3              Sortie (Feature Map)
+---+---+---+---+       +---+---+---+           +---+---+
| 1 | 2 | 3 | 0 |       | 1 | 0 | 1 |           | 7 | 6 |
| 0 | 1 | 2 | 1 |   *   | 0 | 1 | 0 |    =      | 4 | 5 |
| 1 | 0 | 1 | 2 |       | 1 | 0 | 1 |           +---+---+
| 2 | 1 | 0 | 1 |       +---+---+---+
+---+---+---+---+
```

Calcul : somme des produits élément par élément

- Position (0,0) : 1*1* + *2*0 + 3*1* + *0*0 + 1*1* + *2*0 + 1*1* + *0*0 + 1*1 = 7

Hyperparamètres :

- **Taille du filtre** : 3x3, 5x5, 7x7 (3x3 le plus courant)
- **Stride** : pas de déplacement (1 = pixel par pixel, 2 = saute 1 pixel)
- **Padding** : ajout de zeros autour (preserve la dimension)

- **Nombre de filtres** : détermine la profondeur de sortie (ex: 64, 128, 256)

Ce que détectent les filtres :

- Couches initiales : bords horizontaux, verticaux, diagonaux
- Couches intermediaires : textures, motifs répétitifs
- Couches profondes : parties d'objets (yeux, roues, etc.)

2. Fonction d'activation (ReLU)

Formule : ReLU(x) = max(0, x)

```
Avant ReLU          Après ReLU
+----+----+         +----+----+
| -2 |  3 |         |  0 |  3 |
| -1 |  5 |    ->   |  0 |  5 |
+----+----+         +----+----+
```

Pourquoi ReLU ?

- Introduit la **non-linéarité** (sinon empilement de convolutions = une seule convolution)
- Calcul rapide (simple comparaison)
- Atténue le problème du vanishing gradient (gradient = 1 si x > 0)

2.1. Introduit la non-linéarité

Pourquoi c'est nécessaire ?

Sans fonction d'activation, une convolution est une opération linéaire. Si on empile plusieurs couches linéaires :

Couche 1 : $y1 = W1 * x$
Couche 2 : $y2 = W2 * y1 = W2 * W1 * x = W_{combine} * x$
Mathématiquement, n couches linéaires = 1 seule couche linéaire. Le réseau ne peut alors apprendre que des relations linéaires (lignes droites, plans).

Comment ReLU résout ça ?

ReLU "casse" la linéarité en introduisant une condition :

Si x > 0 : sortie = x (linéaire)
Si x <= 0 : sortie = 0 (écrase)
Cette simple condition permet au réseau d'apprendre des frontières de décision courbes et des représentations complexes (visages, objets, textures).

2.2. Calcul rapide

Pourquoi c'est important ?

Un CNN traite des millions de valeurs par image. Chaque opération compte.

Comment ReLU est rapide ?

Comparons avec d'autres fonctions d'activation :

Fonction	Operation	Complexite
ReLU	if x > 0 then x else 0	1 comparaison
Sigmoid	1 / (1 + exp(-x))	1 division + 1 exponentielle
Tanh	(exp(x) - exp(-x)) / (exp(x) + exp(-x))	2 exponentielles + 1 division

ReLU est 10-100x plus rapide car c'est juste une comparaison a zéro, opération native du CPU/GPU.

2.3. Atténue le vanishing gradient

Pourquoi c'est un problème ?

Pendant l'entraînement, le gradient se propage de la sortie vers l'entrée (backpropagation). Avec sigmoid/tanh :

Dérivée de sigmoid : f'(x) = f(x) * (1 - f(x))
Valeur max : 0.25 (quand x = 0)
A chaque couche, le gradient est multiplié par une valeur < 1. Après 10 couches : 0.25^10 = 0.00000095 -> le gradient devient quasi-nul.

Les premières couches n'apprennent plus rien.

Comment ReLU résout ça ?

Dérivée de ReLU :

- Si x > 0 : gradient = 1 (passe intact)
- Si x <= 0 : gradient = 0 (bloque)
  Quand x > 0, le gradient n'est pas réduit. Il peut traverser de nombreuses couches sans disparaître, permettant aux réseaux profonds (50, 100+ couches) de s'entraîner efficacement.

3. Pooling (sous-échantillonnage)

Max Pooling 2x2 : garde la valeur maximale de chaque région

```
Entrée 4x4                                                    Sortie 2x2
+----+----+----+----+                                         +----+----+
|  1 |  3 |  2 |  1 |      Region 1        Region 2          |  4 |  6 |
+----+----+----+----+      (haut-gauche)   (haut-droite)     +----+----+
|  2 |  4 |  1 |  6 |      +----+----+     +----+----+       |  3 |  4 |
+----+----+----+----+      |  1 |  3 |     |  2 |  1 |       +----+----+
|  1 |  2 |  3 |  4 |      |  2 |  4 |     |  1 |  6 |
+----+----+----+----+      +----+----+     +----+----+
|  0 |  3 |  2 |  1 |      max = 4         max = 6
+----+----+----+----+
                            Region 3        Region 4
                            (bas-gauche)    (bas-droite)
                            +----+----+     +----+----+
                            |  1 |  2 |     |  3 |  4 |
                            |  0 |  3 |     |  2 |  1 |
                            +----+----+     +----+----+
                            max = 3         max = 4
```

Objectifs :

- **Réduction dimensionnelle** : divise par 2 la taille spatiale
- **Invariance locale** : petit déplacement du motif = même sortie
- **Réduction du sur-apprentissage** : moins de paramètres

Architecture type d'un CNN

```
Input (224x224x3)
    |
[Conv 3x3, 64] -> ReLU -> [Conv 3x3, 64] -> ReLU -> MaxPool 2x2
    |
    v (112x112x64)
[Conv 3x3, 128] -> ReLU -> [Conv 3x3, 128] -> ReLU -> MaxPool 2x2
    |
    v (56x56x128)
[Conv 3x3, 256] -> ReLU -> [Conv 3x3, 256] -> ReLU -> MaxPool 2x2
    |
    v (28x28x256)
    ...
    |
Flatten -> FC-4096 -> ReLU -> Dropout -> FC-1000 -> Softmax
    |
    v
Probabilités pour 1000 classes
```

Calcul du nombre de paramètres

Couche de convolution :

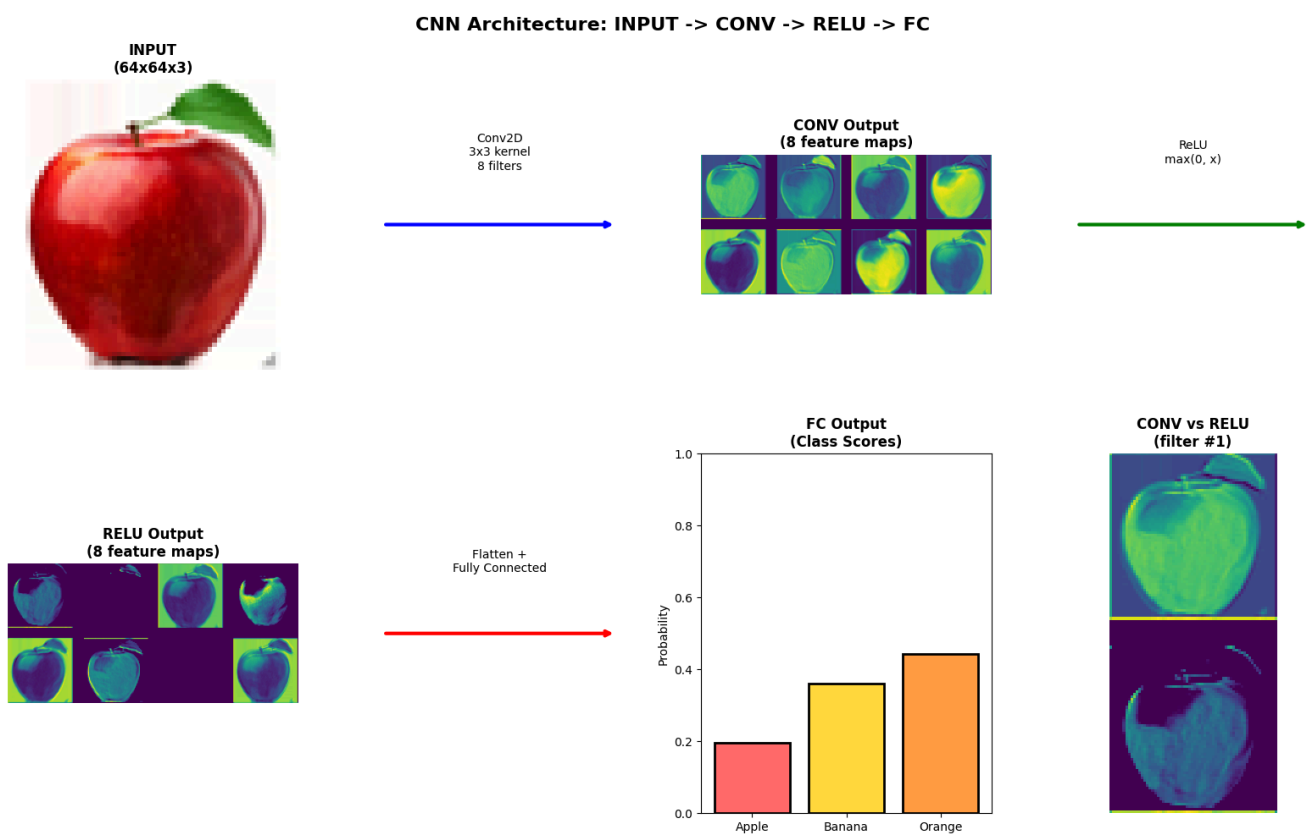
- Paramètres = (K x K x C_in + 1) x C_out
- Exemple : Conv 3x3, 64 -> 128 filtres = (3 x 3 x 64 + 1) x 128 = 73,856

Couche fully-connected :

- Paramètres = (N_in + 1) x N_out
- Exemple : 7x7x512 -> 4096 = (25,088 + 1) x 4096 = 102,764,544

Résumé : Pipeline CNN

Etape	Rôle	Effet
Convolution	Extraction de features	Détecte motifs locaux
ReLU	Non-linéarité	Permet représentations complexes
Pooling	Réduction spatiale	Invariance + compression
FC	Classification	Décision finale



Cet exemple illustre très bien l'intérêt de l'entraînement

4 Contexte et motivation

Rappel : Évolution historique

- **1998** : LeNet-5 (LeCun) - 5 couches, reconnaissance de chiffres
- **2012** : AlexNet - 8 couches, victoire ImageNet (top-5 error : 15.3%)
- **2014** : VGGNet - 16-19 couches
- **2015** : ResNet - jusqu'à 152 couches
- **2017** : MobileNet - optimisation pour l'embarqué

***Top-5 error** : Métrique de classification où le modèle prédit 5 classes les plus probables. L'erreur est comptée seulement si la vraie classe n'est pas parmi ces 5 prédictions. Utilisée sur ImageNet (1000 classes) car certaines catégories sont très proches (ex: races de chiens).*

4.1 LeNet-5 (LeCun, 1998)

- **Principe** : Premier CNN moderne conçu pour la reconnaissance de chiffres manuscrits (MNIST). Introduit les concepts fondamentaux : convolutions locales, partage de poids et sous-échantillonnage spatial.
- **Architecture** :
  - Input : 32x32x1 (image en niveaux de gris)
  - C1 : Conv 5x5, 6 filtres -> 28x28x6
  - S2 : Average Pooling 2x2 -> 14x14x6
  - C3 : Conv 5x5, 16 filtres -> 10x10x16
  - S4 : Average Pooling 2x2 -> 5x5x16
  - C5 : Conv 5x5, 120 filtres -> 1x1x120
  - F6 : FC 84 neurones
  - Output : FC 10 classes (chiffres 0-9)
  - Total : ~60,000 paramètres

C = Convolution (couche convolutive)
- C1, C3, C5 sont des couches de convolution
S = Subsampling (sous-échantillonnage)
- S2, S4 sont des couches de pooling (average pooling dans LeNet)
- Le terme "subsampling" était utilisé par LeCun en 1998, aujourd'hui on dit "pooling"
F = Fully connected (couche dense)
- F6 est la couche entièrement connectée

- **Limitation** :
  - Activation tanh/sigmoid (problème du gradient vanishing)

    - Average pooling moins performant que max pooling
    - Trop peu profond pour des tâches complexes (ImageNet)
    - Capacité limitée : ne fonctionne bien que sur des images simples basse résolution

4.2 Limites des CNN simples (type AlexNet)

- **Principe** :
    - Architecture séquentielle simple : couches empilées les unes après les autres
    - Schéma typique : [Conv -> ReLU -> Pool] x N -> Flatten -> [FC -> ReLU] x M -> Softmax
    - Augmentation progressive des canaux (ex: 96 -> 256 -> 384 -> 256) pour capturer des features de plus en plus abstraites
    - Réduction progressive de la résolution spatiale via pooling (227 -> 55 -> 27 -> 13 -> 6)
    - Couches FC massives en fin de réseau (4096 -> 4096 -> 1000) concentrant la majorité des paramètres
- **Profondeur limitée** :
    - Au-delà de 10-15 couches, l'entraînement devient instable
    - Le gradient "s'évanouit" ou "explose" lors de la rétropropagation
- **Problème du Vanishing Gradient** :
    - Lors de la rétropropagation, les gradients sont multipliés à chaque couche
    - Si les poids sont < 1, le gradient tend vers 0 exponentiellement
    - Les premières couches n'apprennent plus (gradients quasi-nuls)
    - Formellement : gradient final proportionnel à produit(Wi) avec Wi < 1
- **Dégradation des performances** :
    - Paradoxe observé : un réseau plus profond peut avoir une erreur PLUS élevée
    - Ce n'est pas du sur-apprentissage (erreur training aussi plus élevée)
    - C'est un problème d'optimisation, pas de capacité
- **Coût computationnel** :
    - Nombre de paramètres croît avec la profondeur
    - Mémoire GPU limitée
    - Temps d'entraînement prohibitif

Objectifs des architectures avancées

- **Profondeur** : permettre des centaines de couches
- **Stabilité** : garantir la convergence de l'entraînement
- **Efficacité** : optimiser le ratio précision/calcul
- **Déploiement** : adapter aux contraintes matérielles

4.3 VGG - Simplicité et profondeur (2014, Simonyan & Zisserman)

Idée clé

- **Question posée** : peut-on améliorer les performances en augmentant simplement la profondeur ?
- **Réponse** : oui, avec des filtres 3x3 uniquement

Principe architectural

- **Convolutions uniformes 3x3** :
  - Stride = 1, padding = 1 (conserve la dimension spatiale)
  - Deux conv 3x3 = champ réceptif équivalent à une conv 5x5
  - Trois conv 3x3 = champ réceptif équivalent à une conv 7x7
  - Avantage : plus de non-linéarités (ReLU) + moins de paramètres
- **Structure en blocs** :
  - Conv3x3 -> ReLU -> Conv3x3 -> ReLU -> MaxPool 2x2
  - Doublement des filtres après chaque pooling (64 -> 128 -> 256 -> 512)

Champ réceptif (Receptive Field) : Zone de l'image d'entrée qui influence un neurone donné dans une couche profonde. Plus on empile de couches, plus le champ réceptif s'agrandit. Avec VGG, 2 convolutions 3x3 donnent un champ réceptif de 5x5, et 3 convolutions 3x3 donnent 7x7, tout en utilisant moins de paramètres qu'un filtre 7x7 direct ((3x3x3=27) vs (7x7=49)).

Variantes principales

Modèle	Couches	Paramètres	Top-5 Error ImageNet
VGG-11	11	133M	10.4%
VGG-16	16	138M	7.4%
VGG-19	19	144M	7.3%

Architecture VGG-16 détaillée

```
Input: 224x224x3
  |
[Conv3x3-64] x2 -> MaxPool -> 112x112x64
[Conv3x3-128] x2 -> MaxPool -> 56x56x128
[Conv3x3-256] x3 -> MaxPool -> 28x28x256
[Conv3x3-512] x3 -> MaxPool -> 14x14x512
```

```
[Conv3x3-512] x3 -> MaxPool -> 7x7x512
  |
Flatten -> FC-4096 -> FC-4096 -> FC-1000 -> Softmax
```

Avantages

- **Simplicité** : architecture régulière et prévisible
- **Transfert learning** : excellentes features pré-entraînées
- **Référence** : benchmark pour comparer de nouvelles architectures

Limites

- **138M paramètres** : majoritairement dans les couches FC (102M)
- **Mémoire** : ~500 MB pour le modèle seul
- **Inférence** : ~20 images/sec sur GPU moderne
- **Non adapté** : déploiement mobile ou temps réel

Cas d'usage : prototypage, benchmarks académiques, extraction de features

---

4.4 ResNet - Connexions résiduelles (2015, He et al.)

Problème adressé : la dégradation

- **Observation expérimentale** :
  - ResNet-56 sans skip : 27% d'erreur
  - ResNet-56 avec skip : 6% d'erreur
  - Un réseau de 56 couches performait MOINS bien qu'un réseau de 20 couches
- **Cause** : difficulté d'optimisation, pas de sur-apprentissage

Innovation clé : le bloc résiduel

- **Idée** : au lieu d'apprendre H(x), apprendre F(x) = H(x) - x
- **Sortie** : y = F(x) + x (connexion de saut / skip connection)

```
          x ──────────────────────┐
          |                       |
          v                       |
     [Conv 3x3]                   |
          |                       |
     [BatchNorm]                  |
```

```
          |                       |
       [ReLU]                     |
          |                       |
     [Conv 3x3]                   |
          |                       |
     [BatchNorm]                  |
          |                       |
          v                       v
         (+) ←———————————————————┘
          |
       [ReLU]
          |
          v
         y = F(x) + x
```

Pourquoi ça fonctionne ?

- **Gradient direct** : le gradient peut "sauter" les couches via la connexion identité
- **Optimisation facilitée** : apprendre F(x) = 0 est plus facile qu'apprendre H(x) = x
- **Pas de paramètres supplémentaires** : l'addition est gratuite

Batch Normalization (BN)

Technique de normalisation des activations intermédiaires, omniprésente dans les CNN modernes.

Principe : Pour chaque mini-batch, on normalise les activations puis on applique une transformation apprise :

```
            x - moyenne(x)
  x_norm = ——————————————
            sqrt(var(x) + e)

  y = gamma * x_norm + beta
```

- `gamma` et `beta` : paramètres appris (échelle et décalage)
- `e` : petite constante pour la stabilité numérique

Avantages :

- **Accélère la convergence** : permet des learning rates plus élevés
- **Régularisation implicite** : réduit le besoin de dropout
- **Stabilise l'entraînement** : évite les explosions/disparitions de gradient
- **Réduit la dépendance à l'initialisation** des poids

En inférence : on utilise les moyennes/variances calculées sur l'ensemble d'entraînement (running average)

Types de blocs

Type	Utilisé dans	Structure
Basic Block	ResNet-18/34	Conv3x3 -> BN -> ReLU -> Conv3x3 -> BN
Bottleneck	ResNet-50/101/152	Conv1x1 -> Conv3x3 -> Conv1x1 (réduction de dimension)

Bottleneck Block (ResNet-50+)

```
x (256 canaux)
    |
[Conv 1x1, 64] ——— Reduction
    |
[Conv 3x3, 64] ——— Traitement
    |
[Conv 1x1, 256] —— Expansion
    |
   (+) + x
```

- **Avantage** : moins de paramètres qu'un bloc basique équivalent

Variantes principales

Modèle	Couches	Paramètres	Top-5 Error	GFLOPs
ResNet-18	18	11.7M	10.9%	1.8
ResNet-34	34	21.8M	9.4%	3.6
ResNet-50	50	25.6M	7.1%	3.8
ResNet-101	101	44.5M	6.4%	7.6
ResNet-152	152	60.2M	6.2%	11.5

Avantages

- **Profondeur extrême** : jusqu'à 1000+ couches possible
- **Entraînement stable** : convergence garantie
- **Transfer learning** : pré-entraîné sur ImageNet, excellent pour fine-tuning

- **Standard industriel** : backbone par défaut pour détection/segmentation

Limites

- **Complexité** : architecture moins intuitive que VGG
- **Inférence** : ~50-100 images/sec selon variante
- **Mémoire** : activations intermédiaires coûteuses

Cas d'usage : vision industrielle, imagerie médicale, détection d'objets, recherche

4.5 MobileNet – Efficacité et embarqué

Contexte et motivation

- **Contraintes des systèmes embarqués** :
  - CPU mobile : 10-100x plus lent qu'un GPU serveur
  - Mémoire limitée : 1-4 GB RAM
  - Batterie : consommation énergétique critique
  - Latence : besoin de temps réel (<100ms)
- **Objectif** : réduire drastiquement le coût de calcul tout en préservant la précision

Innovation clé : Convolutions séparables en profondeur

Convolution standard :

- Filtre K x K appliqué sur TOUS les canaux simultanément
- Coût : $K \times K \times C_{in} \times C_{out}$ opérations par pixel

Convolution séparable = Depthwise + Pointwise

Depthwise (convolution en profondeur)

Applique **un filtre $K \times K$ par canal d'entrée**, indépendamment.

- Chaque canal est traité séparément
- Pas de mélange d'information entre canaux
- Coût : $K \times K \times C_{in}$ opérations par pixel

```
Input: H x W x C_in
    |
[Filtre K x K] x C_in  (1 filtre par canal)
    |
Output: H x W x C_in
```

Pointwise (convolution 1x1)

Convolution avec des filtres de taille **1x1** pour combiner les canaux.

- Mélange l'information entre canaux
- Change le nombre de canaux ($C_{in} \to C_{out}$)
- Coût : $C_{in} \times C_{out}$ opérations par pixel

```
Input: H x W x C_in
    |
[Filtre 1x1 x C_in] x C_out
    |
Output: H x W x C_out
```

Gain total : Depthwise + Pointwise = $K \times K \times Cin + C_{in} \times C_{out}$ vs $K \times K \times C_{in} \times C_{out}$ (standard)

Pour K=3, C_in=C_out=256 : réduction de **~9x** du coût de calcul

```
Convolution Standard (coûteuse)
================================
Input: H x W x C_in
    |
[Filtre K x K x C_in] x C_out
    |
Output: H x W x C_out

Coût: K^2 x C_in x C_out x H x W


Convolution Séparable (économique)
==================================
Input: H x W x C_in
    |
[Depthwise: K x K x 1] x C_in    <- 1 filtre par canal
    |
Intermediate: H x W x C_in
    |
[Pointwise: 1 x 1 x C_in] x C_out  <- mélange des canaux
    |
Output: H x W x C_out

Coût: K^2 x C_in x H x W + C_in x C_out x H x W
```

Réduction du coût de calcul

- **Ratio de réduction** : (K^2 x C_out + C_out) / (K^2 x C_in x C_out)
- **Avec K=3** : réduction d'environ 8-9x le nombre d'opérations
- **Exemple concret** :
  - Standard 3x3, 256->256 : 589,824 multiplications
  - Séparable 3x3, 256->256 : 68,096 multiplications

Hyperparamètres de contrôle

- **Width Multiplier (alpha)** : réduit le nombre de canaux
  - alpha = 1.0 : modèle complet
  - alpha = 0.5 : 50% des canaux (4x moins de calcul)
  - alpha = 0.25 : 25% des canaux
- **Resolution Multiplier (rho)** : réduit la taille d'entrée
  - 224 x 224 (standard)
  - 192 x 192, 160 x 160, 128 x 128

Versions de MobileNet

Version	Année	Innovation principale	Top-1 ImageNet
MobileNetV1	2017	Depthwise separable	70.6%
MobileNetV2	2018	Inverted residuals + Linear bottleneck	72.0%
MobileNetV3	2019	Neural Architecture Search (NAS)	75.2%

Top-1 ImageNet

Métrique de performance standard pour la classification d'images.

- **Définition** : pourcentage d'images où la classe prédite (probabilité max) correspond à la vraie classe
- **ImageNet** : dataset de 1.2M images, 1000 classes
- Top-1 = 75% signifie que le modèle prédit correctement 75% des images
- Variante **Top-5** : la bonne classe est parmi les 5 prédictions les plus probables

Neural Architecture Search (NAS)

Méthode d'**optimisation automatique** de l'architecture d'un réseau.

- Un algorithme (RL, évolution génétique, gradient) explore l'espace des architectures possibles
- Chaque architecture candidate est entraînée et évaluée

- L'objectif : trouver le meilleur compromis précision/latence/taille
- MobileNetV3 utilise NAS pour optimiser la structure des blocs et les hyperparamètres

Avantage : dépasse souvent les architectures conçues manuellement
Inconvénient : coût de recherche très élevé (milliers de GPU-heures)

Inverted residuals + Linear bottleneck

Innovations clés de **MobileNetV2** :

Inverted Residual (résiduel inverse) :

- Contrairement à ResNet qui compresse puis expand (256->64->256)
- MobileNetV2 fait l'inverse : expand puis compresse (64->384->64)
- La connexion résiduelle relie les couches à **faible dimension**

```
ResNet:      large -> petit -> large  (bottleneck classique)
MobileNetV2: petit -> large -> petit  (inverted bottleneck)
```

Linear Bottleneck :

- Pas de ReLU après la dernière convolution 1x1
- Raison : ReLU détruit l'information dans les espaces de faible dimension
- Préserver l'information avec une activation linéaire (identité)

Comparaison des performances

Modèle	Paramètres	MAdd	Latence (ms)	Top-1 Acc
VGG-16	138M	15.5G	523	71.5%
ResNet-50	25.6M	3.8G	147	76.0%
MobileNetV2	3.4M	300M	23	72.0%
MobileNetV3-Small	2.5M	56M	6	67.4%

Latence mesurée sur Pixel 1 (CPU)

***MAdd** (Multiply-Add) : nombre d'opérations multiplication-addition effectuées par le réseau pour traiter une image. Une convolution implique des multiplications suivies d'additions, comptées ensemble. Mesure standard du coût de calcul (300M = 300 millions d'opérations, 15.5G = 15.5 milliards)*

Avantages

- **Ultra-léger** : 3-5 MB vs 500 MB pour VGG
- **Temps réel** : 30+ FPS sur smartphone
- **Flexible** : ajustable via alpha et rho
- **TensorFlow Lite** : optimisé pour le déploiement mobile

Limites

- **Précision** : 4-6% inférieure à ResNet sur ImageNet
- **Tâches complexes** : moins performant sur détection fine
- **Quantification** : sensible à la compression INT8

Cas d'usage : applications mobiles, drones, caméras embarquées, IoT, AR/VR

4.5 Comparaison synthétique des architectures

Critère	LeNet-5	AlexNet	VGG-16	ResNet-50	MobileNetV2
Paramètres	60K	62M	138M	25.6M	3.4M
Taille modèle	~0.2 MB	~240 MB	~530 MB	~100 MB	~14 MB
GFLOPs	0.0004	0.7	15.5	3.8	0.3
Top-1 ImageNet	N/A (MNIST)	63.3%	71.5%	76.0%	72.0%
Latence GPU	N/A	~200 img/s	~20 img/s	~100 img/s	~300 img/s
Latence CPU mobile	Temps réel	Lent	Inutilisable	~7 img/s	~30 img/s
Transfer learning	Non	Moyen	Excellent	Excellent	Bon
Déploiement edge	Oui	Non	Non	Difficile	Oui

---

5. Transfer Learning

5.1 Problématique de l'entraînement

Entraînement from scratch (à partir de zéro)

Entraîner un CNN depuis zéro nécessite :

- **Enorme dataset** : ImageNet = 1.2M images, 1000 classes
- **Ressources GPU** : ResNet-50 ~ 14 jours sur 1 GPU, quelques heures sur 8 GPUs

- **Expertise** : choix des hyperparamètres (learning rate, batch size, augmentation)

Problème : la plupart des projets n'ont pas ces ressources ni assez de données.

Pourquoi le Transfer Learning ?

Les premières couches d'un CNN apprennent des **features génériques** :

- Couches 1-2 : edges, couleurs, textures simples
- Couches intermédiaires : motifs, formes géométriques
- Couches profondes : features spécifiques au dataset (visages, voitures, etc.)

Idée clé : réutiliser les features génériques apprises sur un grand dataset (ImageNet) et les adapter à une tâche spécifique.

5.2 Modèles pré-entraînés

Définition

Un modèle **pré-entraîné** est un réseau dont les poids ont déjà été optimisés sur un large dataset (généralement ImageNet).

Sources principales

Framework	Commande	Modèles disponibles
PyTorch	`torchvision.models`	ResNet, VGG, MobileNet, EfficientNet, ViT...
TensorFlow/Keras	`tf.keras.applications`	Mêmes modèles + NASNet, Xception...
Hugging Face	`timm` library	700+ modèles pré-entraînés

Exemple PyTorch

```python
import torchvision.models as models

# Charger ResNet-50 pré-entraîné sur ImageNet
model = models.resnet50(weights='IMAGENET1K_V2')

# Charger MobileNetV3 pré-entraîné
model = models.mobilenet_v3_small(weights='IMAGENET1K_V1')
```

5.3 Stratégies de Transfer Learning

***Learning Rate** : Taux d'apprentissage qui contrôle l'amplitude des mises à jour des poids lors de l'entraînement. En transfer learning, on utilise généralement un learning rate **faible** (ex: 1e-4 à 1e-5) pour préserver les features pré-apprises, contre un learning rate standard (1e-2 à 1e-3) pour un entraînement from scratch.*

Stratégie 1 : Feature Extraction (extracteur de features)

On **gèle** toutes les couches convolutives et on remplace uniquement la dernière couche (classificateur).

```
[Couches convolutives GELÉES] -> [Nouvelle couche FC]
         (poids fixes)              (poids entraînés)
```

Quand l'utiliser :

- Petit dataset (<1000 images)
- Dataset similaire à ImageNet (objets, animaux, scènes)
- Ressources limitées

```python
# Geler toutes les couches
for param in model.parameters():
    param.requires_grad = False

# Remplacer le classificateur (ResNet)
model.fc = nn.Linear(2048, num_classes)
```

Stratégie 2 : Fine-tuning (ajustement fin)

On **dégèle** certaines couches (généralement les dernières) et on les ré-entraîne avec un faible learning rate.

```
[Couches basses GELÉES] -> [Couches hautes DÉGELÉES] -> [Nouveau classificateur]
   (features génériques)     (features adaptées)          (nouvelle tâche)
```

Quand l'utiliser :

- Dataset de taille moyenne (1000-10000 images)
- Dataset différent d'ImageNet (images médicales, satellite, etc.)

```python
# Geler les premières couches (layer1, layer2)
for name, param in model.named_parameters():
    if 'layer1' in name or 'layer2' in name:
```

```
        param.requires_grad = False

# Learning rate plus faible pour le fine-tuning
optimizer = torch.optim.Adam(model.parameters(), lr=1e-5)
```

Stratégie 3 : Fine-tuning progressif (Gradual Unfreezing)

Dégeler les couches **progressivement**, des plus profondes aux plus basses.

1. Entraîner le classificateur (1-2 epochs)
2. Dégeler la dernière couche convolutive (2-3 epochs)
3. Dégeler la couche précédente (2-3 epochs)
4. Continuer jusqu'à convergence

Avantage : évite de "casser" les features génériques des premières couches.

5.4 Bonnes pratiques

Learning rate

Stratégie	Learning rate recommandé
Feature extraction	1e-3 à 1e-2 (classifieur seul)
Fine-tuning	1e-5 à 1e-4 (10-100x plus faible)
Couches basses	1e-6 (si dégelées)

Data augmentation

Essentielle pour éviter l'overfitting sur petits datasets :

- **Géométrique** : rotation, flip, crop, scale
- **Photométrique** : brightness, contrast, saturation
- **Avancée** : Mixup, CutMix, RandAugment

```
transforms = T.Compose([
    T.RandomResizedCrop(224),
    T.RandomHorizontalFlip(),
    T.ColorJitter(brightness=0.2, contrast=0.2),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225])  # Stats ImageNet
])
```

Normalisation

Toujours utiliser les mêmes statistiques de normalisation que le modèle pré-entraîné (moyennes et écarts-types ImageNet).

5.5 Domain Adaptation

Cas où le domaine source (ImageNet) est très différent du domaine cible.

Domaine cible	Difficulté	Stratégie
Photos naturelles	Facile	Feature extraction suffit
Images médicales (X-ray, IRM)	Moyenne	Fine-tuning complet
Images satellite/aérien	Moyenne	Fine-tuning + augmentation spécifique
Microscopie, spectroscopie	Difficile	Pré-entraînement sur dataset intermédiaire

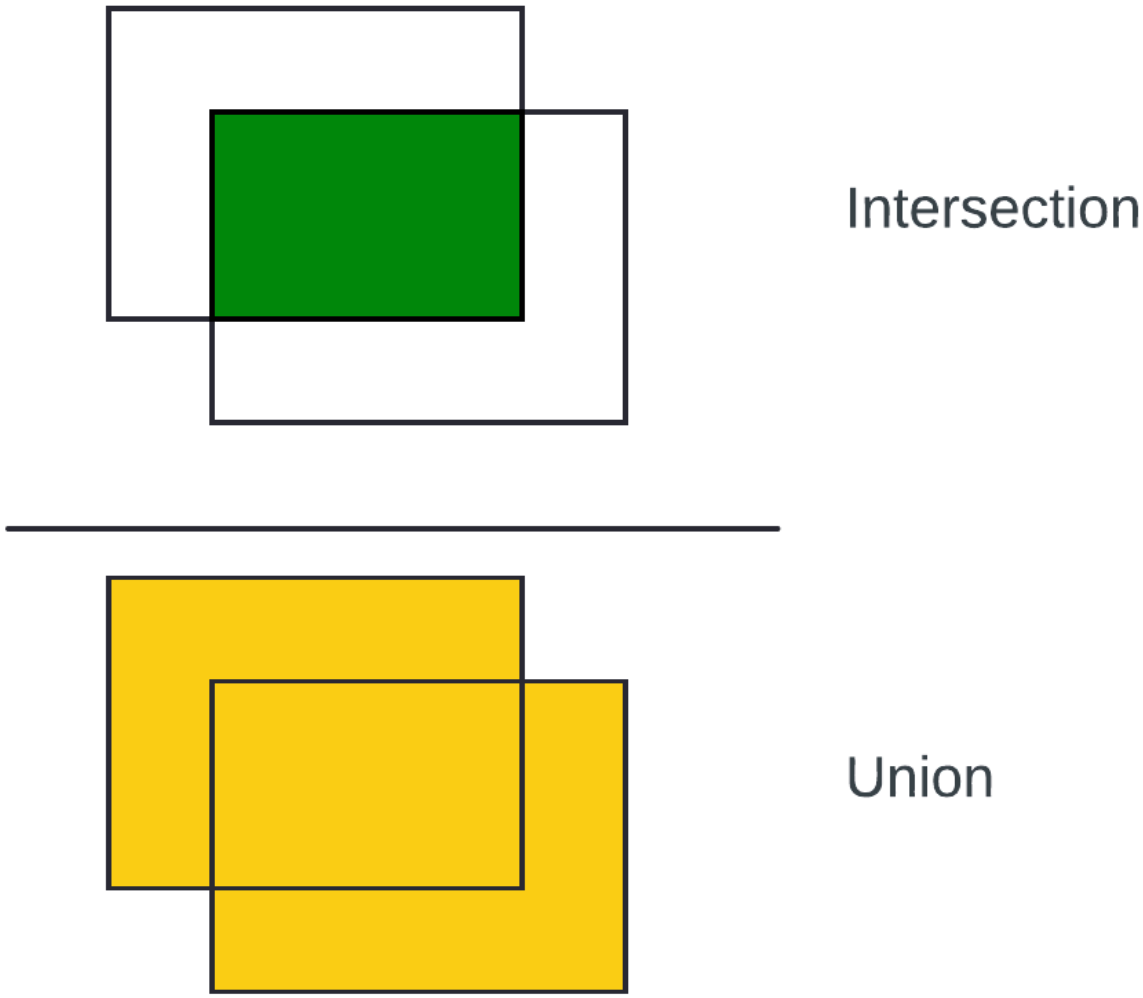
5.6 Résumé : arbre de décision

```
                      Taille du dataset ?
                            |
              +-------------+-------------+
              |                           |
          < 1000 images              > 10000 images
              |                           |
      Feature Extraction            Fine-tuning complet
      (geler tout sauf FC)          (toutes couches)
              |                           |
              +-------------+-------------+
                            |
                  1000 - 10000 images
                            |
                  Fine-tuning partiel
                (dégeler couches hautes)
```

---

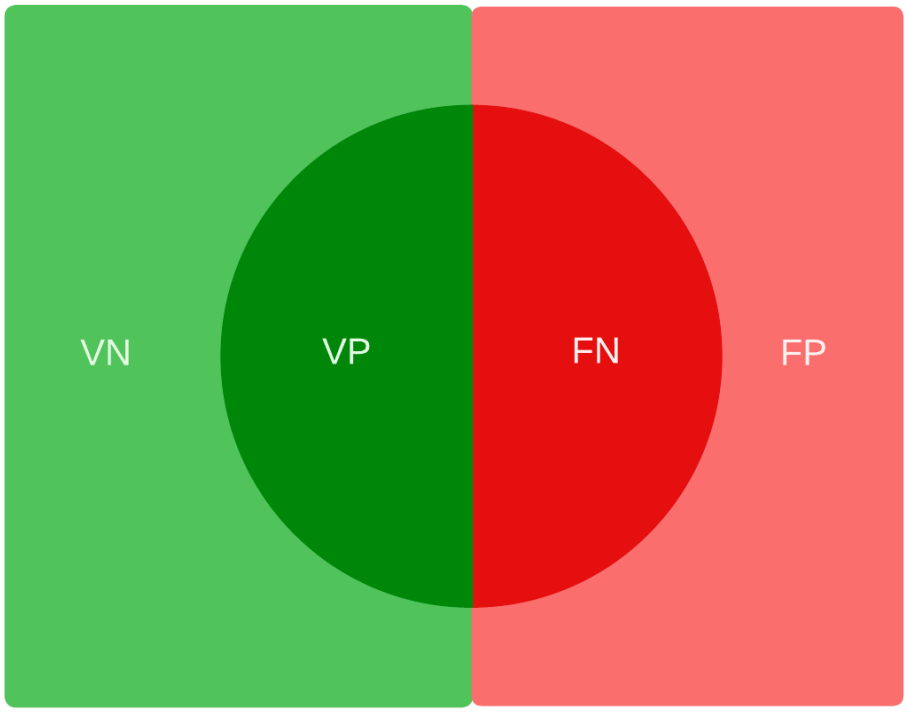
6. Critères de choix d'une architecture

Concepts préalables :



- **IoU** (Intersection over Union) : mesure le chevauchement entre la boîte prédite et la vérité terrain
  - IoU = (aire intersection) / (aire union)

- Seuil typique : IoU >= 0.5 pour considérer une détection correcte



- **Précision (Precision)** : parmi les détections, combien sont correctes ? $\frac{TP}{TP+FP}$
- **Rappel (Recall)** : parmi les objets réels, combien sont détectés ? $\frac{TP}{TP+FN}$
- **F1-score** : Combinaison de précision et rappel $\frac{TP}{TP+\frac{1}{2}(FP+FN)}$

6.1 Métriques de précision

- **Top-1 Accuracy** : prédiction correcte = classe la plus probable
- **Top-5 Accuracy** : classe correcte dans les 5 premières prédictions
- **mAP (détection)** : mean Average Precision

mAP (mean Average Precision)

*Métrique standard pour évaluer les modèles de **détection d'objets**.*

***Calcul du mAP** :*

1. *Pour chaque classe, tracer la courbe Precision-Recall*
2. *Calculer l'aire sous cette courbe = **AP** (Average Precision) de la classe*
3. ***mAP** = moyenne des AP sur toutes les classes**

***Variantes** :*

- ***mAP@0.5** : IoU seuil à 0.5 (PASCAL VOC)*

- ***mAP@0.5:0.95** : moyenne sur plusieurs seuils IoU (COCO, plus strict)*

Compromis précision/efficacité

Domaine	Exigence précision	Architecture recommandée
Médical (diagnostic)	Critique (>95%)	ResNet-101/152
Industrie (contrôle qualité)	Elevée (>90%)	ResNet-50
Retail (comptage)	Modérée (>80%)	MobileNetV2
Mobile (filtres AR)	Acceptable (>70%)	MobileNetV3-Small

Impact du Transfer Learning

- **Pré-entraîné ImageNet** : +10-20% de précision sur petits datasets
- **Fine-tuning** : adapter les dernières couches au domaine cible
- Tous les modèles disponibles pré-entraînés sur PyTorch/TensorFlow

---

6.2 Temps d'inférence

Contraintes selon l'application

Application	Latence max	FPS minimum
Vidéo temps réel	33 ms	30 FPS
Robotique	100 ms	10 FPS
Batch processing	-	-
Application mobile	100 ms	10 FPS

Facteurs influents

- **Matériel** :
  - GPU serveur (V100, A100) : 100-1000x plus rapide que CPU
  - GPU embarqué (Jetson) : 10-50x plus rapide que CPU mobile
  - NPU/TPU mobile : optimisé pour les CNN
- **Optimisations** :
  - Quantification INT8 : 2-4x accélération
  - Pruning : réduction des poids inutiles

- TensorRT / CoreML : compilation optimisée

6.3 Taille du modèle et déploiement

Contraintes mémoire

Plateforme	RAM disponible	Taille max modèle
Serveur	16-256 GB	Illimitée
Desktop	8-32 GB	~1 GB
Mobile	2-8 GB	~100 MB
IoT/Edge	512 MB - 2 GB	~50 MB
Microcontrôleur	<1 MB	~100 KB

Considérations déploiement

- **Stockage** : taille du fichier modèle
- **Bande passante** : mise à jour OTA (Over-The-Air)
- **Temps de chargement** : démarrage de l'application
- **Consommation énergétique** : autonomie batterie

6.4 Arbre de décision

```
                      [Choix architecture CNN]
                              |
              +---------------+---------------+
              |                               |
       [Déploiement ?]                 [Recherche/Proto]
              |                               |
     +-------+-------+                      VGG
     |               |
[Serveur]      [Embarqué]
     |               |
ResNet-50      +----+----+
               |         |
           [Précision   [Temps réel
            critique]    prioritaire]
               |              |
```

```
            ResNet-18     MobileNetV3
```

6.5 Synthèse décisionnelle

Situation	Choix recommandé	Justification
Précision critique, serveur GPU	ResNet-101/152	Meilleure accuracy
Production industrielle	ResNet-50	Bon compromis
Application mobile	MobileNetV2/V3	Léger et rapide
Prototype académique	VGG-16	Simple à comprendre
IoT / Edge computing	MobileNetV3-Small	Ultra-léger
Détection d'objets	ResNet-50 + FPN	Backbone standard

Conclusion

Points clés à retenir :

1. **VGG** : architecture pédagogique, trop lourde pour la production
2. **ResNet** : standard industriel, skip connections révolutionnaires
3. **MobileNet** : convolutions séparables, déploiement embarqué
4. **Choix** : toujours un compromis précision/vitesse/taille